

计算思维与人工智能

实验四 指针和字符串

TAKE YOU ON A TRIP
GO!

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

探索 从未停止

实验目的

- 理解字符串的存储本质极其与指针的关联关系
- 熟练运用指针和字符串处理函数进行字符串操作
- 掌握函数间传递字符串的方法，并理解地址传递的机制

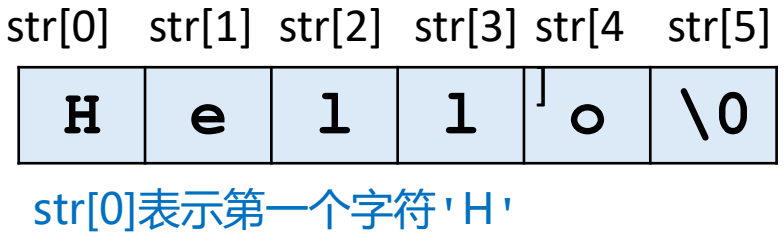
指针和字符串

C语言没有提供**字符串**数据类型，通过**字符数组**间接实现，以**特殊字符‘\0’**（空字符）作为结束标志（ASCII 码为 0）

➢ 单个字符串

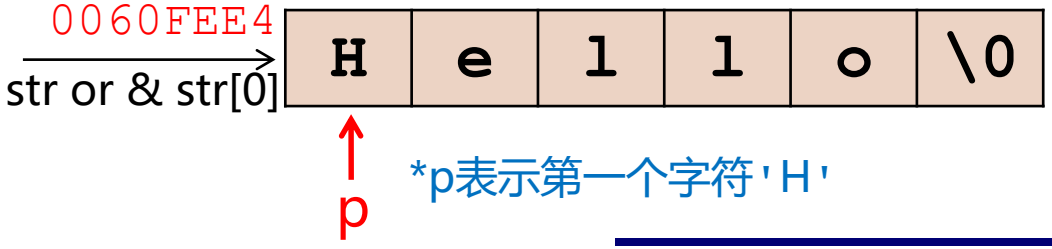
- **存储**：一维字符数组

```
char str[6] = "Hello";
```



- **遍历**：可使用下标 or **字符指针**

```
char *p = str;
while (*p != '\0') {
    printf("%c", *p);
    p++;      //通过指针偏移可访问后续字符
}
```



指针和字符串

➢ 多个字符串

- **存储**：二维字符数组

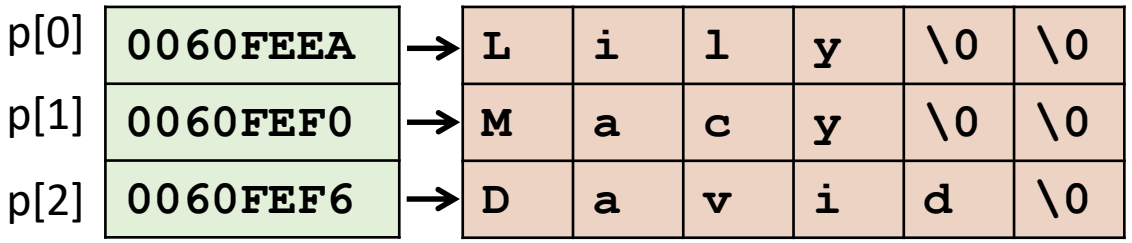
```
char names[3][6] = {"Lily", "Macy",
"David"};
```



- **遍历**：可使用下标 or 字符指针数组

```
for(int i =0; i <3; i++){
    printf("%s\n", names[i]); //使用下标遍历
}

char *p[3];   //使用字符指针数组遍历
for (int i = 0; i < 3; i++) {
    p[i] = names[i]; //等价于 &names[i][0]
}
for (int i = 0; i < 3; i++) {
    printf("%s\n", p[i]);
}
```



指针和字符串

➢ 字符串输入

- **scanf("%s", str)**：直接读取字符串，自动以 \0 结尾，遇到空格、制表符、换行符停止并丢弃
- **getchar()**：逐个字符读取，需手动拼接并添加 \0，能读取空格、换行符等所有字符
- **gets(str)**：直接读取整行字符串（包括空格），自动添加 \0，仅遇到换行符终止，且会丢弃
- **fgets(str, n, stdin)**：推荐的安全方式

  读取 n-1 个字符或遇到换行符停止，自动添加 \0，默认保留换行符（需手动去除）

  n 限制最大读取长度，避免溢出

```c
#include <stdio.h>
#include <stdlib.h>


int main()
{

    char str1[4]={'\0'};
    printf("Please intput str1:\n");
    scanf("%s",&str1);
    printf("str1=%s\n",str1);

    char str2[2][4]={'\0'};
    for(int i = 0; i< 2; i++)
    {
        printf("\nPlease intput str2[%d]:\n",i);
        scanf("%s",&str2[i]);
        printf("str2[%d]=%s\n",i,str2[i]);
    }

    retu
}
```

scanf("%s", str1);

scanf("%s", str2[i]);

```
D:\code\c\2022\stucode\bin\Debug\stucode.exe
Please intput str1:
000000000061FE18    printf("%p\n",str1);
aaa
str1=aaa

Please intput str2[0]:
000000000061FE10    printf("%p\n",str2[i]);
bbb
str2[0]=bbb

Please intput str2[1]:
000000000061FE14
ccc
str2[1]=ccc

Process returned 0 (0x0)   execution time : 9.225 s
Press any key to continue.
```

Logs & others

Code::Blocks

File	Line	Messa
D:\code\c\2...		=== B
		In fu
D:\code\c\2...	10	warni
D:\code\c\2...	10	warni
D:\code\c\2...	17	warni
D:\code\c\2...	17	warni
		=== B

字符串处理函数

```
  char s1[] = "hello";
  char s2[] = "hello";
✗ if (s1 == s2) {
      printf("s1 equals s2\n");
  } //比较的是地址，不是内容!
```

- 字符串不能直接用 = 赋值，而是用strcpy()或strncpy()
- 字符串不能直接用 >、<、== 比较，应该用 strcmp()
- 如果使用字符串处理函数，需要 #include <string.h>

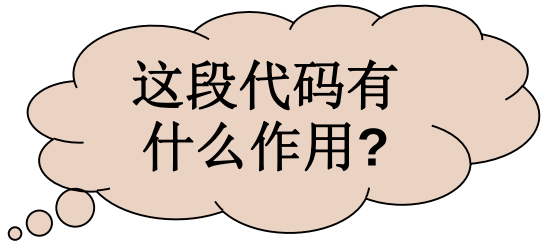
函数	功能	返回值
strlen(const char *str)	计算字符串 str 的长度，不包括结束符 \0	字符串的长度
strcpy(char *dest, const char *src)	将源字符串 src 的内容（含 \0）复制到目标字符串 dest 指向的内存空间	指向目标字符串 dest 的指针
strcmp(const char *str1, const char *str2)	按照字典序（ASCII 码值）比较两个字符串 str1 和 str2	<0、0、>0 分别表示小于、等于、大于
strcat(char *dest, const char *src)	将源字符串 src 的内容追加到目标字符串 dest 的末尾	指向目标字符串 dest 的指针

向函数传递字符串

【例5-1】寻找“最小”字符串

```
#include <stdio.h>
#include <string.h>
#define ARRA_SIZE 80
int main()
{
    int n ;
    char str[ARRA_SIZE], min[ARRA_SIZE];

    printf("Please enter 5 names:\n");
    gets(str);
    strcpy(min,str);
    for(n = 1; n < 5; n++)
    {
        gets(str);
        if(strcmp(str,min) < 0)
        {
            strcpy(min, str);
        }
    }
    printf("The min is:");
    puts(min);
}
```



参考输入输出

```
Please enter 5 names:
abby
lily
bob
shuoe
hdai
The min is:abby
```

这段代码是一个简单的程序，用于从用户输入的五个字符串中找到字典序最小的字符串，并将其输出。

字典序：也称为字母序或词典序，是一种用于对字符串或序列进行排序的方法，类似于字典中单词的排列顺序。

举例：假设有两个字符串 "apple" 和 "banana"。这两个字符串的字典序比较会从左到右逐个字符比较：

在第一个位置上，'a' 在字母表中出现在 'b' 前面，因此 "apple" 的字典序比 "banana" 小。因此，"apple" 应该排在 "banana" 前面。

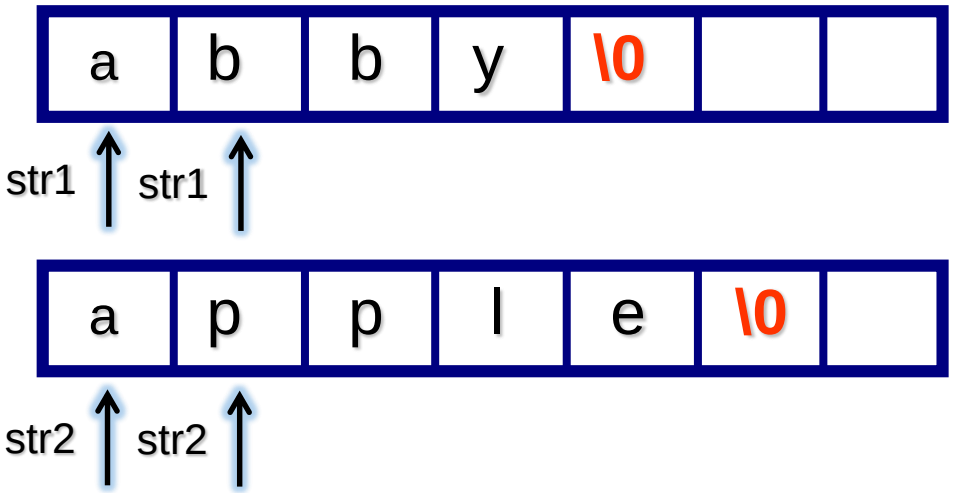
向函数传递字符串

【例5-1】寻找“最小”字符串

使用strcmp函数比较"apple" 和 "abby"的字典序大小

```
int strcmp(const char* str1, const char* str2)
{
          while ((*str1) && (*str1 == *str2))
          {
                    str1++;
                    str2++;
          }


          if (*(unsigned char*)str1 > *(unsigned char*)str2)
          {
                    return 1;
          }
          else if (*(unsigned char*)str1 < *(unsigned char*)str2)
          {
                    return -1;
          }
          else
          {
                    return 0;
          }
}
```



- 第一步*str1==*str2，指针右移
- 第二步*str1!=*str2，跳出循环
- 第三步*(unsigned char*)str1 < *(unsigned char*)str2，返回-1

向函数传递字符串

【例5-1】寻找“最小”字符串

strcmp函数实现：

```
int strcmp(const char* str1, const char* str2)
{
        while ((*str1) && (*str1 == *str2))
        {
                str1++;
                str2++;
        }

        if (*(unsigned char*)str1 > *(unsigned char*)str2)
        {
                return 1;
        }
        else if (*(unsigned char*)str1 < *(unsigned char*)str2)
        {
                return -1;
        }
        else
        {
                return 0;
        }
}
```

- 首先比较两个串的第一个字符（本质上是比较两个字符的ASCII码），如果相等则接着比较下一个字符，直到两个字符不相等或者一个字符串遇到结束符'/0'为止
- 接下来比较停止位置p的两个字符ASCII码大小
  - 若*str1>*str2，则返回1
  - 若*str1<*str2，则返回-1
  - 若*str1==*str2，则返回0（说明两个字符串完全相同）
- 因为\0字符的ASCII码是0，若一个字符串是另一个字符串的子串，则长的字符串字典序更大

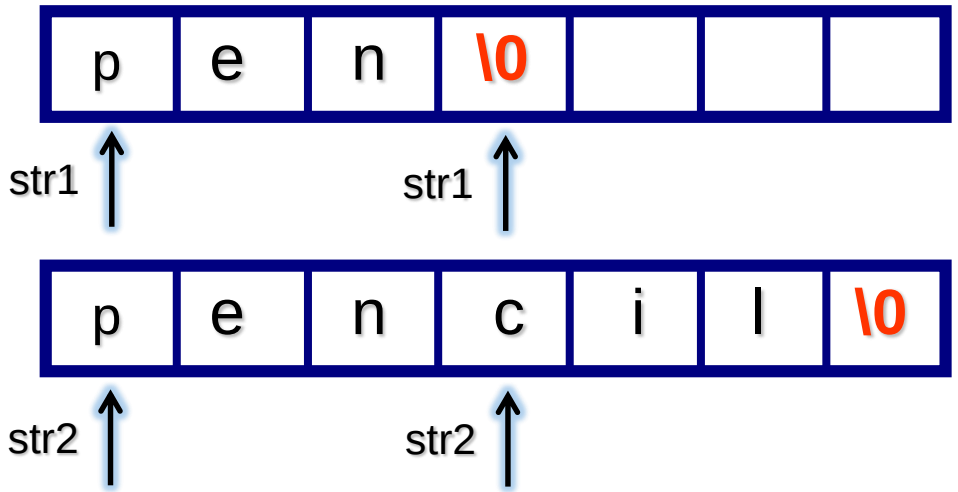
向函数传递字符串

【例】寻找“最小”字符串

使用strcmp函数比较"pen" 和 "pencil"的字典序大小

```
int strcmp(const char* str1, const char* str2)
{
          while ((*str1) && (*str1 == *str2))
          {
                    str1++;
                    str2++;
          }


          if (*(unsigned char*)str1 > *(unsigned char*)str2)
          {
                    return 1;
          }
          else if (*(unsigned char*)str1 < *(unsigned char*)str2)
          {
                    return -1;
          }
          else
          {
                    return 0;
          }
}
```



- 第一步*str1==*str2，指针右移3次
- 第二步*str1!=*str2，跳出循环（'\0'的ASCII码为0）
- 第三步*(unsigned char*)str1 < *(unsigned char*)str2，返回-1

向函数传递字符串

【例5-2】字符串逆序存放

用户输入的一个字符串，利用指针编程将字符串逆序存储

```c
#include <stdio.h>
#include <string.h>
#define MAX_SIZE 100
int main() {
    char str[MAX_SIZE];
    char *p, *q, temp;
    puts("Please enter a string:");
    gets(str);
    p = str;
    q = str + strlen(str) - 1;
    while (p < q) {
        temp = *p;
        *p = *q;
        *q = temp;
        p++;
        q--;
    }
    puts(str);
    return 0;
}
```

这段代码有什么作用?

参考输入输出

```
Please enter a string:
HelloWorld
dlroWolleH
```

这段代码是一个简单的程序，用于利用指针将用户输入的一个字符串进行原地逆序存储，并将其输出。

字符串逆序：是指将字符序列的排列顺序完全颠倒，即将首字符与尾字符、次首字符与次尾字符依次交换，直到中间位置。

举例：假设有一个字符串 "abcde"。逆序的过程通常通过两个指针分别指向字符串的开头和结尾来进行：

Step1：开头的字符 'a' 与结尾的字符 'e' 交换，字符串变为 "ebcda"

Step2：指针向中间移动，次开头的字符 'b' 与次结尾的字符 'd' 交换，字符串变为 "edcba"

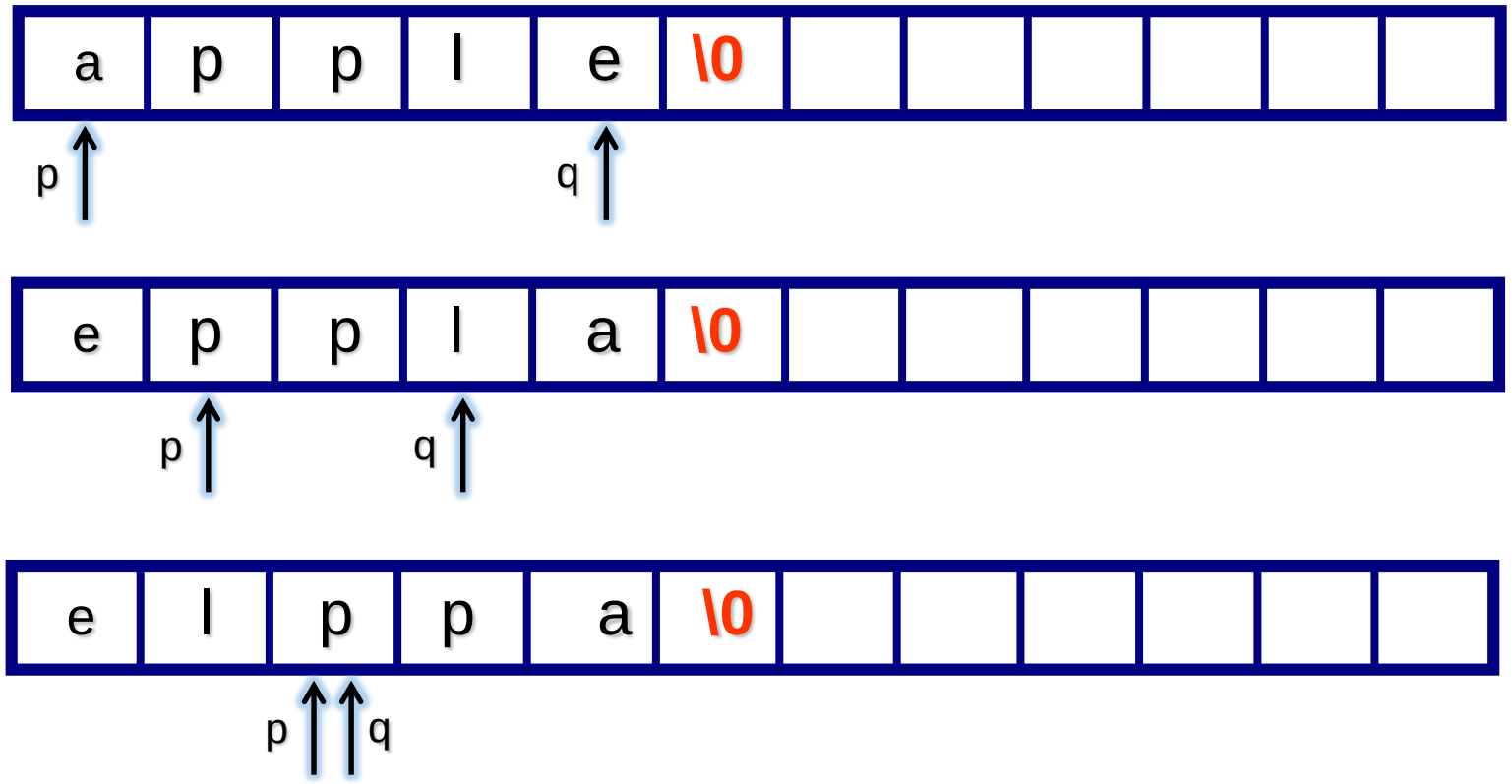
Step3：指针相遇时操作结束，原始字符串 "abcde" 最终变成了 "edcba"

向函数传递字符串

【例5-2】字符串逆序存放

用户输入的一个字符串，利用指针编程将字符串逆序存储

```
#include <stdio.h>
#include <string.h>
#define MAX_SIZE 100
int main() {
    char str[MAX_SIZE];
    char *p, *q, temp;
    puts("Please enter a string:");
    gets(str);
    p = str;
    q = str + strlen(str) - 1;
    while (p < q) {
        temp = *p;
        *p = *q;
        *q = temp;
        p++;
        q--;
    }
    puts(str);
    return 0;
}
```

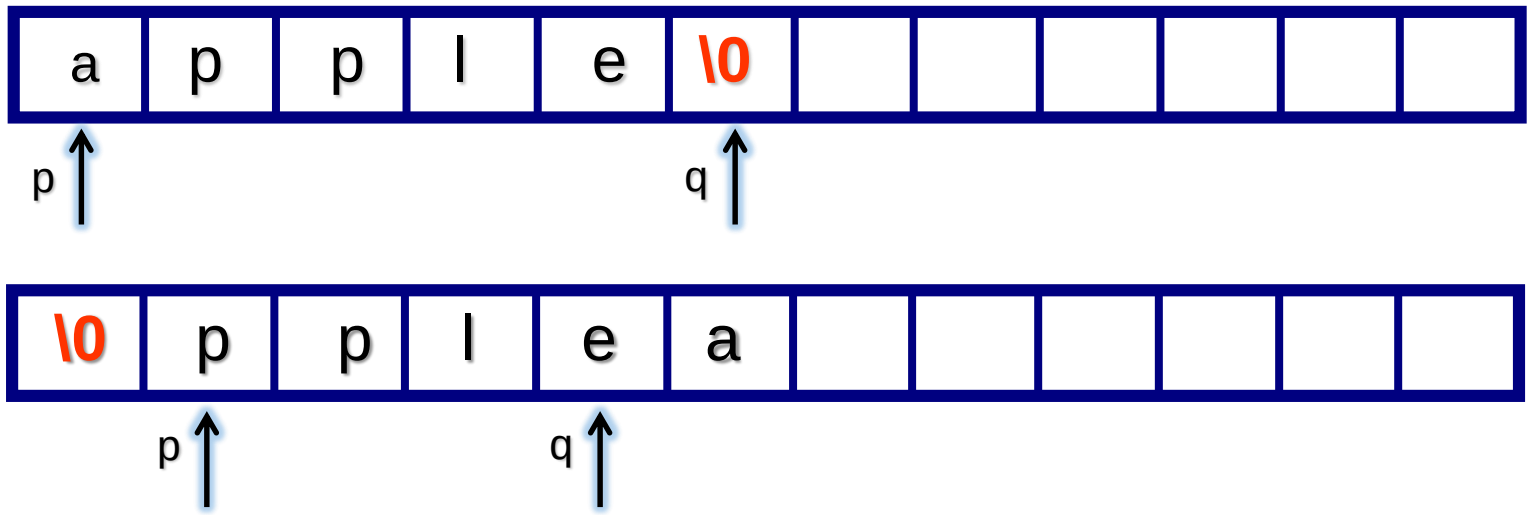


向函数传递字符串

【例5-2】字符串逆序存放

若将代码第10行改为q = str +strlen(str)，即去掉-1，会发生什么?

```
#include <stdio.h>
#include <string.h>
#define MAX_SIZE 100
int main() {
    char str[MAX_SIZE];
    char *p, *q, temp;
    puts("Please enter a string:");
    gets(str);
    p = str;
    q = str + strlen(str) - 1;
    while (p < q) {
        temp = *p;
        *p = *q;
        *q = temp;
        p++;
        q--;
    }
    puts(str);
    return 0;
}
```



第一次交换后，\0 被换到了字符串开头（str[0] 变成了 \0）。 当 puts(str) 时，一读到开头的 \0 就会认为字符串结束了，导致输出为空。

实验内容

4001 大数相加

使用一维数组实现两个正整数相加，输出它们的和。

注：两个加数的位数不一定相等

【输入描述】

两行，分别是两个正整数

位数最少为二十位，不超过五十位

【输出描述】

两个正整数的和

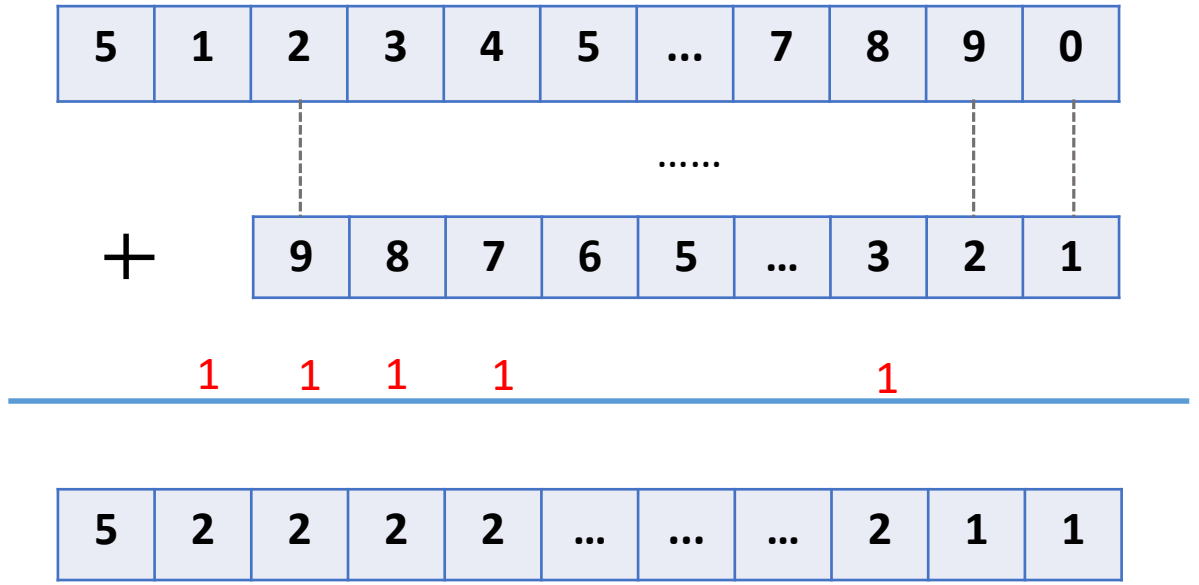
【输入示例】

12345678934134123456 7

11135439503091234567

【输出示例】

13459222884443246913 4



实验内容

4002 单词统计器

输入一行英文字符串，请统计并输出其中不同单词的数量。单词由字母组成，大小写不敏感（如 "Hello" 与 "hello" 视为相同）；单词之间可能由空格或标点隔开。注：请使用指针和字符串函数实现。

【输入描述】	【输入示例】
一行，英文文本	**Hello, world! This is a world of hello.**
【输出描述】	**【输出示例】**
不同单词的数量	**6**

Hello
world
This
is
a
of

实验内容

4003 密码翻译

在情报传递过程中，为了防止情报被截获，往往需要对情报用一定的方式加密，简单的加密算法虽然不足以完全避免情报被破译，但仍然能防止情报被轻易的识别。

我们给出一种最简的的加密方法，对给定的一个字符串，把其中从a-y，A-Y的字母用其后继字母替代，把z和Z用a和A替代，其他非字母字符不变，则可得到一个简单的加密字符串。

【输入描述】

输入一行，包含一个字符串，长度小于80个字符。

【输出描述】

输出每行字符串的加密字符串。

【输入示例】

Hello! How are you!

【输出示例】

Ifmmp! Ipx bsf zpv!

课程平台使用指南

- 登录网址：http://10.250.48.240:9000，推荐浏览器：Chrome；
- 初始用户名、密码均为学号，登录后请修改；
- 提交截止时间内，可以重新提交作业，不限次数；
- 只要有一次提交通过，即使重新提交代码未通过，也视为access状态；
- 目前仅支持C99，请在本地IDE完成调试后再提交课程平台；
- 代码没有点击“提交”之前，所有内容没有保存，请做好本地备份；
- 代码“提交”，需刷新之后，才可以查看“提交记录”和“已提交代码“；
- 只支持单文件；

课程平台使用指南

- 常见oj错误提示：

① Compile Error　编译错误，不符合语法规范

② Wrong Output　表示运行结果不正确

③ Representation error　表示输出格式不正确

④ Runtime error　请检查是否有除零、数组越界、eles语句块没有用{}括起来等

⑤ Nonzero Exit Status　一般表示主函数返回非0值



FEEL THE MEANING OF THE TRIP

请同学们开始实验

TAKE YOU ON A TRIP

GO!

哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

探索 从未停止